Figure 7.8 The interprocess communication between Alpha App and Beta App is based on a DNS name that gets resolved to one of the instance IP addresses by a load-balancer component. The service registration process is handled by the platform transparently.

The Kubernetes implementation of this service discovery pattern is based on Service objects. A *Service* is "an abstract way to expose an application running on a set of Pods as a network service" (https://kubernetes.io/docs/reference/glossary).

A Service object is an abstraction targeting a set of Pods (typically using labels) and defining the access policy. When an application needs to contact a Pod exposed by a Service object, it can use the Service name instead of calling the Pod directly. That's what you did to let the Catalog Service application interact with the PostgreSQL instance (`polar-postgres` was the name of the Service exposing the PostgreSQL Pod). The Service name is then resolved to the IP address of the Service itself by a local DNS server running in the Kubernetes Control Plane.

NOTE The IP address assigned to a Service is fixed for its lifetime. Therefore, the DNS resolution of a Service name doesn't change as often as it would with application instances.

After resolving the Service name to its IP address, Kubernetes relies on a proxy (called *kube-proxy*), which intercepts the connection to the Service object and forwards the request to one of the Pods targeted by the Service. The proxy knows all the replicas available and adopts a load-balancing strategy depending on the type of Service and the proxy configuration. There is no DNS resolution involved in this step, solving the problems I mentioned earlier. The service discovery implementation adopted by Kubernetes is shown in figure 7.9.